

MODELOS AVANZADOS DE COMPUTACIÓN

Relación 4

1. (a) Un grafo dirigido se dice que es *acíclico* si no tiene ciclos. Demostrar que todo grafo finito dirigido acíclico tiene una *fuentes* (un nodo al que no llegan arcos).
(b) Utilizar (a) para demostrar que un grafo dirigido con n nodos es acíclico si y solo si se pueden numerar los nodos del 1 al n de manera que siempre los arcos van desde números más pequeños a números más grandes.
(c) Utilizar (a) y (b) para describir un algoritmo en tiempo polinómico para determinar cuando un grafo dirigido es acíclico.
2. (a) Demostrar que un grafo es *bipartito* (se puede dividir en dos partes, que no son necesariamente iguales, de manera que todas las aristas van de una parte a otra) si y solo si todos sus ciclos son de longitud par.
(b) Utilizar (a) para describir un algoritmo en tiempo polinómico para comprobar si un grafo es bipartito.
3. Demostrar que $\mathbf{P}$ es cerrada para la unión y la intersección.
4. Sea C una clase de funciones de los enteros no negativos en los enteros no negativos. Se dice que C es cerrada para la composición polinómica por la izquierda si $f(n) \in C$ implica que $p(f(n)) = O(g(n))$ para alguna función $g(n) \in C$, y donde $p(n)$ es un polinomio cualquiera. Se dice que C es cerrada para la composición polinómica por la derecha si $f(n) \in C$ implica que $f(p(n)) = O(g(n))$ para alguna función $g(n) \in C$, y donde $p(n)$ es un polinomio cualquiera.

Determinar cuales de las siguientes clases de funciones son cerradas para la composición polinómica por la derecha y cuales lo son por la izquierda.

- (a) $\{n^k : k > 0\}$
- (b) $\{n.k : k > 0\}$
- (c) $\{k^n : k > 0\}$
- (d) $\{2^{n^k} : k > 0\}$
- (e) $\{\log^k(n) : k > 0\}$
- (f) $\{\log(n)\}$

Nota: en los dos últimos casos se entiende que las funciones se redondean al entero más cercano. En vez de considerar todos los polinomios, como estamos hablando de órdenes

de complejidad será suficiente considerar los polinomios de la forma $p(n) = n^l$, donde l es un número natural.

5. Sea $L = \{wcw : w \in \{0,1\}^*\}$, Demostrar que L se puede reconocer en espacio $\log(n)$.
6. Demostrar que si L esta en $\mathbf{P}$, entonces L^* también está en $\mathbf{P}$.
7. Demostrar que si L esta en $\mathbf{NP}$, entonces L^* también está en $\mathbf{NP}$.
8. Demostrar que $\mathbf{NP}$ es cerrada para la unión y la intersección.
9. Demostrar que si $\mathbf{NP} \neq \mathbf{CoNP}$ entonces $\mathbf{P} \neq \mathbf{NP}$
10. Supongamos que una entrada es una palabra de paréntesis. Demostrar que determinar si están emparejados y anidados correctamente está en $\mathbf{L}$. Lo están $((()))$ y $((()))$, pero no lo están $)()()$ ni $((()))()$.
11. Consideremos el problema anterior, pero ahora con dos tipos de paréntesis $(,)$ y $[,]$. Demostrar que determinar si están bien escritos también está en $\mathbf{L}$. Aquí $([()])$ está permitido, pero $([])$ no. Demostrar que si la entrada se lee de izquierda a derecha sin volver hacia atrás, requieren $O(n)$ de memoria.
12. Demostrar que el problema del palíndromo, requiere $O(n)$ de memoria si no puede volver hacia atrás.
13. Demostrar que $\mathbf{NP} \neq \mathbf{ESPACIO}(n)$. Nota: Este ejercicio es muy complicado. Como sugerencia, para probar que las clases son diferentes, probar que $\mathbf{NP}$ cumple la propiedad de ser cerrada para reducciones en $\mathbf{L}$ (si $P_1 \propto P_2$ y P_2 está en $\mathbf{NP}$, entonces P_1 está en $\mathbf{NP}$), mientras que $\mathbf{ESPACIO}(n)$, no.

1. (a) Un grafo dirigido se dice que es *acíclico* si no tiene ciclos. Demostrar que todo grafo finito dirigido acíclico tiene una *fuentes* (un nodo al que no llegan arcos).

(b) Utilizar (a) para demostrar que un grafo dirigido con n nodos es acíclico si y solo si se pueden numerar los nodos del 1 al n de manera que siempre los arcos van desde números más pequeños a números más grandes.

(c) Utilizar (a) y (b) para describir un algoritmo en tiempo polinómico para determinar cuando un grafo dirigido es acíclico.

Sea M una MT.

1) Recibe un grafo dirigido $G=(V,E)$ como entrada.

2) Calcula grado de entrada $\forall v \in V$. Inicializa $k=1$ para numerar. $\|O(|E|)$

3) $L = \{v \in V \mid \text{in-degree}(v)=0\}$ $\|O(|V|)$

while ($L \neq \emptyset$) $\|O(|V|)$

Tomar $v \in L$. $L = L \setminus \{v\}$

Asignar(v, k)

$k++$

$\forall u \in V \mid (v,u) \in E \quad \| \leq O(|E|) \quad \sum_{v \in V} \text{out-degree}(v) = |E|$. Máximo $|E|$ iteraciones en total

$\text{in-degree}(u) --$

if ($\text{in-degree}(u)=0$) $L = L \cup \{u\}$

if ($k=|V|+1$) aceptar
else rechazar

$$T(n) \leq O(|V|+|E|) \leq O(|V|^2)$$

2. (a) Demostrar que un grafo es *bipartito* (se puede dividir en dos partes, que no son necesariamente iguales, de manera que todas las aristas van de una parte a otra) si y solo si todos sus ciclos son de longitud par.

(b) Utilizar (a) para describir un algoritmo en tiempo polinómico para comprobar si un grafo es bipartito.

Búsqueda en anchura. Sea M una MT

1) recibe grafo $G=(V,E)$, $|V|=n$, $|E|=m$

2) Inicializa $L=\emptyset$ y $particion(v)=-1 \forall v \in V$ (no visitados). $//O(n)$
 $particion(v)=i$ indica estar en la partición i del grafo bipartito.

3) $\forall v \in V$

if ($particion(v) \neq -1$)

$particion(v)=0$

$L=L \cup \{v\}$

while ($L \neq \emptyset$) $//O(n)$

Extraer $u \in L$. $L=L \setminus \{u\}$

$\forall w \in V / (u,w) \in E$ $//O(m)$ total

if ($particion(w) \neq -1$)

$particion(w)=1-particion(u)$

$L=L \cup \{w\}$

else if ($particion(w) = particion(u)$) rechazar

acceptar

$$T(n) \leq O(n+m) \leq O(n^2)$$

3. Demostrar que P es cerrada para la unión y la intersección.

Sean $L_1, L_2 \in P \Rightarrow \exists M_1, M_2 \text{ MTD}, p_1, p_2 \text{ pol.} / L_i \text{ se acepta en } p_i(n) \text{ pasos } i \in \{1, 2\}$

Sea M una MT con el siguiente comportamiento:

1) Recibe $x \in A^* / |x| = n$ como entrada

2) Copia x en otra cinta. $O(n)$

3) Simula M_1 con x . Si M_1 rechaza $\Rightarrow M$ lo hace. En caso contrario, ir a 4).

4) Simula M_2 con x . Si M_2 acepta $\Rightarrow M$ lo hace. En caso contrario, rechaza.

Vemos $L(M) = L_1 \cap L_2$, que acepta en tiempo $T(n) \leq O(n) + p_1(n) + p_2(n)$
 $\Rightarrow L_1 \cap L_2 \in P$

Sea M' una MT con el siguiente comportamiento:

1) Recibe $x \in A^* / |x| = n$ como entrada

2) Copia x en otra cinta. $O(n)$

3) Simula M_1 con x . Si M_1 acepta $\Rightarrow M'$ lo hace. En caso contrario, ir a 4).

4) Simula M_2 con x . Si M_2 acepta $\Rightarrow M'$ lo hace. En caso contrario, rechaza.

Vemos $L(M') = L_1 \cup L_2$, que acepta en tiempo $T(n) \leq O(n) + p_1(n) + p_2(n)$
 $\Rightarrow L_1 \cup L_2 \in P$

4. Sea C una clase de funciones de los enteros no negativos en los enteros no negativos. Se dice que C es cerrada para la composición polinómica por la izquierda si $f(n) \in C$ implica que $p(f(n)) = O(g(n))$ para alguna función $g(n) \in C$, y donde $p(n)$ es un polinomio cualquiera. Se dice que C es cerrada para la composición polinómica por la derecha si $f(n) \in C$ implica que $f(p(n)) = O(g(n))$ para alguna función $g(n) \in C$, y donde $p(n)$ es un polinomio cualquiera.

Determinar cuales de las siguientes clases de funciones son cerradas para la composición polinómica por la derecha y cuales lo son por la izquierda.

Nota: en los dos últimos casos se entiende que las funciones se redondean al entero más cercano. En vez de considerar todos los polinomios, como estamos hablando de órdenes de complejidad será suficiente considerar los polinomios de la forma $p(n) = n^l$, donde l es un número natural.

(a) $\{n^k : k > 0\}$

Sea $g(n) = n^k \in C \Rightarrow p(g(n)) = (n^k)^l = n^{k \cdot l} \in C \Rightarrow$ cerrada por la izda.
 $g(p(n)) = (n^l)^k = n^{l \cdot k} \in C \Rightarrow$ cerrada por la dcha.

(b) $\{n \cdot k : k > 0\}$

Sea $g(n) = n \cdot k \in C \Rightarrow p(g(n)) = (n \cdot k)^l = n^l \cdot k^l \neq O(n) \Rightarrow$ no es cerrada por la izda.
 $g(p(n)) = n \cdot k^l \neq O(n) \Rightarrow$ No es cerrada por la dcha.

(c) $\{k^n : k > 0\}$

Sea $g(n) = k^n \in C \Rightarrow p(g(n)) = (k^n)^l = (k^l)^n = O(k^{n^l}) \quad k^l > 0 \Rightarrow$ es cerrada por la izda.
 $g(p(n)) = k^{n^l} \neq O(k^n) \Rightarrow$ no es cerrada por la dcha.

(d) $\{2^{n^k} : k > 0\}$

Sea $g(n) = 2^{n^k} \Rightarrow p(g(n)) = (2^{n^k})^l = 2^{n^{k \cdot l}} = 2^{n^{k \cdot l}} = 2^{n^{k \cdot l}} = O(2^{n^{k \cdot l}}) \quad k^l > 0 \Rightarrow$ es cerrada por la izda.
 $g(p(n)) = 2^{(n^l)^k} = 2^{n^{l \cdot k}} = O(2^{n^{l \cdot k}}) \quad k^l > 0 \Rightarrow$ cerrada por la dcha.

(e) $\{\log^k(n) : k > 0\}$

Sea $g(n) = \log^k(n) \Rightarrow p(g(n)) = (\log^k(n))^l = \log^{k \cdot l}(n) = O(\log^{k \cdot l}(n)) \quad k^l > 0 \Rightarrow$ cerrada por la izda.
 $g(p(n)) = \log^k(n^l) = l \log^k(n) = O(\log^k(n)) \Rightarrow$ cerrada por la dcha.

(f) $\{\log(n)\}$

Sea $g(n) = \log(n) \Rightarrow p(g(n)) = \log^l(n) \neq O(\log(n)) \Rightarrow$ no es cerrada por la izda.
 $g(p(n)) = \log(n^l) = l \log(n) = O(\log(n)) \Rightarrow$ cerrada por la dcha.

5. Sea $L = \{w^k : w \in \{0,1\}^*\}$. Demostrar que L se puede reconocer en espacio $\log(n)$.

Construimos un NT multicinta que acepta L .

- 1) Recibe $x \in A^*$, $|x| = n$. La cinta de lectura no se modifica.
- 2) Usa cinta de trabajo para guardar la codificación de n $\parallel O(\log(n))$
- 3) Busca c . Si no encuentra rechaza. En caso contrario ir a 4)
- 4) Guarda la posición de c codificada: k . Si $k \neq n - \lfloor \frac{n}{2} \rfloor$ rechaza. $\parallel O(\log(n))$
- 5) Inicializa punteros $P_1 = 0$, $P_2 = k+1$ codificados $\parallel O(\log(n))$
- 6) while($P_1 \neq k$)
 $\text{if}(cinta(P_1) \neq cinta(k)) \text{rechaza}$
 $P_1 = P_1 + 1$, $P_2 = P_2 + 1$ $\parallel$ No consume más espacio, pues $|P_1| = |P_2| = \log(n)$

7) Si sale de 6) acepta

$$S(n) \leq 3 \log(n) \Rightarrow S(n) = O(\log(n))$$

6. Demostrar que si L está en P , entonces L^* también está en P .

$$L^* = \bigcup_{n \in \mathbb{N}} L^n, \quad L^0 = \{\epsilon\}, \quad L^1 = L, \quad L^i = \{w_1 \dots w_i \mid w_k \in L \ \forall k \in \{1 \dots i\}\}$$

Sea M NT que acepta L .

Construimos un M' que acepta L^* :

- 1) Recibe $w = (w(1) \dots w(n)) \in A^*$ / $|w| = n$ como entrada.
 - 2) Inicializa vector booleano $D = (D(0) \dots D(n))$ / $|D| = n+1$
 - 3) $D(0) = \text{true}$ ($\epsilon \in L^*$), $D(i) = \text{false} \ \forall i \in \{1 \dots n\}$ $\parallel O(n)$
 - 4) for($i \in \{1 \dots n\}$) $\parallel O(n)$
 $\text{for}(j \in \{0 \dots i-1\}) \parallel O(n)$
 $\text{if}(D(j) = \text{true} \text{ and } M \text{ acepta } w(j+1 \dots i)) \parallel O(n^2)$
 $D(i) = \text{true}$
 $\text{salir de bucle } j$
- return $D(n)$

$$\text{Por tanto } T(n) \leq O(n) + O(n^2 n^c) = O(n^{c+2}) \Rightarrow L^* \in P$$

7. Demostrar que si L está en NP , entonces L^* también está en NP .

$$L^* = \bigcup_{n \in \mathbb{N}} L^n, \quad L^0 = \{\epsilon\}, \quad L^1 = L, \quad L^i = \{w_1 \dots w_i \mid w_k \in L \forall k \in \{1 \dots i\}\}$$

Sea M MT que acepta L .

Construimos MTND M' que acepta L^* :

- 1) Toma $w \in A^*$ como entrada.
- 2) Elige de forma determinista una partición de $w = w_1 \dots w_k$ $O(n)$
- 3) for $(i \in \{1 \dots k\})$ $k \leq n$
 simular M con w_i $O(p(n))$
 if $(M$ no acepta $w_i)$ M' rechaza
- 4) Si se sale del bucle, M' acepta.

$$\forall n, T(n) \leq O(n) + n p(n) = O(n^k) \quad k > 0 \Rightarrow L^* \in NP$$

8. Demostrar que NP es cerrada para la unión y la intersección.

Sea $L_1, L_2 \in NP \Rightarrow \exists M_1, M_2$ MTND que aceptan L_1, L_2 en tiempo $p_1(n), p_2(n)$.

Construimos MT M' que acepta $L_1 \cup L_2$:

- 1) Recibe $x \in A^*$ $|x| = n$ como entrada
- 2) Copia x en otra cinta. $O(n)$
- 3) Simula M_1 y M_2 con x en serie. $O(p_1(n) + p_2(n)) = O(p'(n))$, p' pol.
- 4) Si M_1, M_2 aceptan $\Rightarrow M'$ lo hace.

$$\text{Veamos } T(n) \leq n + p_1(n) + p_2(n) = O(p'(n)) \quad p' \text{ pol.} \Rightarrow T(n) = O(n^k) \quad k > 0 \\ \Rightarrow L_1 \cup L_2 \in NP$$

Construimos MT M'' que acepta $L_1 \cap L_2$:

- 1) Recibe $x \in A^*$ $|x| = n$ como entrada
- 2) Copia x en otra cinta. $O(n)$
- 3) Elige de forma no determinista simular M_1 y M_2 con x . $O(p_1(n)) = O(p_2(n))$ para algún $i \in \{1, 2\}$ p' pol.
- 4) Si M_i acepta $\Rightarrow M''$ lo hace.

$$\text{Veamos } T(n) \leq n + \max_{i \in \{1, 2\}} \{p_i(n)\} = O(p'(n)) \quad p' \text{ pol.} \Rightarrow T(n) = O(n^k) \quad k > 0 \\ \Rightarrow L_1 \cap L_2 \in NP$$

9. Demostrar que si $NP \neq CoNP$ entonces $P \neq NP$

Probamos por contrarrecíproco.

$$P = NP \Rightarrow CoP = CoNP \Rightarrow CoNP = CoP = P = NP$$

↑
clases deterministas

10. Supongamos que una entrada es una palabra de paréntesis. Demostrar que determinar si están emparejados y anidados correctamente está en **L**. Lo están $((()))$ y $((()))$, pero no lo están $)() ($ ni $((())) ($.

Construimos **M** MT:

- 1) Recibe $x \in A^*$, $|x| = n$ como entrada $O(\log(n))$
- 2) Mantiene 1 cuenta auxiliar con un contador.
- 3) Lee cada símbolo de x . Si lee $($, incrementa el contador. Si lee $)$, decrementa. Si contador < 0 rechaza. $O(\log(n))$
- 4) Si al acabar de leer x contador $= 0$, acepta.

Vemos $S(n) \leq 2O(\log(n)) \Rightarrow S(n) = O(\log(n)) \Rightarrow L(M) \in L$

11. Consideremos el problema anterior, pero ahora con dos tipos de paréntesis $(,)$ y $[,]$.

Demostrar que determinar si están bien escritos también está en **L**. Aquí $(([]))$ está permitido, pero $([])$ no. Demostrar que si la entrada se lee de izquierda a derecha sin volver hacia atrás, requieren $O(n)$ de memoria.

A)

Sea **M** una MT:

- 1) Recibe $x \in A^*$ $|x| = n$ como entrada
- 2) Usamos contadores: $O(\log(n))$

- * i : posición actual
- * j : índice para retroceder
- * saltos:
- * balance-global

```

3) for (i = 1 to n) {
    if (x(i) = '(' || '[') balance-global++  $O(\log(n))$ 
    if (x(i) = ')' || ']') {
        balance-global--  $O(\log(n))$ 
        if (balance-global < 0) rechazar
        for (j = i - 1 to 1) {
            if (x(j) = '(' || '[') saltos++  $O(\log(n))$ 
            if (x(j) = ')' || ']') {
                saltos--  $O(\log(n))$ 
                if (saltos = 0) {
                    if (x(i) = '(' & x(j) != '[') || (x(i) = '[' & x(j) != '(') rechazar
                    break
                }
            }
        }
    }
}

```

acceptar

Vemos $S(n) \leq O(\log(n)) \Rightarrow L(M) \in L$

B)

Sea M una MT que no puede retroceder en la cinta de entrada.

- 1) Recibe $x \in A^* / |x| = n$ como entrada.
- 2) Usa cinta auxiliar a modo de pila para recordar símbolos
- 3) $j = 0$

```

for (i = 1 to n) { // O(n)
    if (x(i) = '(' || '[') {
        j++
        cinta(j) = x(i) // O(1)
    }
    else if (x(i) = ')' || ']')
        if (j = 0) rechazar // cierre sin apertura
        if (cinta(j) != '(' ^ x(i) = ')') ∨ (cinta(j) != '[' ^ x(i) = ']') rechazar
    }
}
if (j = 0) aceptar
else rechazar
    
```

Vemos $S(n) = O(n)$

12. Demostrar que el problema del palíndromo, requiere $O(n)$ de memoria si no puede volver hacia atrás.

Sea M una MT que no vuelve hacia atrás en la cinta de entrada.

- 1) Recibe $x \in A^* / |x| = n$ como entrada
- 2) Utiliza cinta auxiliar como pila.

```

3) j = 0
for (i = 1 to ⌊n/2⌋) // O(n/2) = O(n)
    j++
    cinta(j) = x(i) // O(1)
}

for (i = ⌊n/2⌋ + 1 to n)
    if (x(i) != cinta(j)) rechazar
    j--
}
aceptar
    
```

Vemos $S(n) = O(n)$